

**PROJECT REPORT OF INDUSTRY ORIENTED HANDS-ON EXPERIENCE
(IOHE)**

ON

Civic Eye - a Grievance Redressal and Complaint Management System

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Pratyukt Nag

[2210990679]

Supervised By:

Dr. Shikha Tuteja

Mentor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND
TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
1	Declaration	1
2	Acknowledgement	2
3	Introduction to Civic Eye	3
4	Problem Statement	4
4.1	Abstract	4
4.2	Tools and Technology	5
4.3	Methodology	7
4.4	Implementation	9
5	Appendices	11

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **Civic Eye a Grievance Redressal and Complaint Management System** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Shikha Tuteja**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura
Date: 13-05-2026

Pratyukt Nag
2210990679

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Shikha Tuteja
Associate Professor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

It is my pleasure to be indebted to various people, who directly or indirectly contributed to the development of this work and who influenced my thinking, behaviour, and acts during my study.

I express our sincere gratitude to all for providing me an opportunity to undergo Integrated Project as part of the curriculum.

I am thankful to **Dr. Shikha Tuteja** for her support, cooperation, and motivation provided to us during the training for constant inspiration, presence, and blessings.

Lastly, I would like to thank the almighty and our parents for their moral support and friends with whom I shared my day-to-day experience and received lots of suggestions that improve my quality of work.

INTRODUCTION TO CIVIC-EYE

The development of **Civic-Eye – a Grievance Redressal and Complaint Management System** – is built upon modern web development frameworks to ensure security, scalability, reliability, and an intuitive user experience. Since grievance management systems involve multiple stakeholders such as citizens, department officials, and super-administrators, implementing a structured and framework-driven architecture is essential for maintaining efficiency, transparency, and controlled access across all user levels.

On the frontend, Civic-Eye utilizes **React.js** combined with **Tailwind CSS**, enabling a component-based and responsive user interface. This approach ensures modularity, reusability, and seamless performance across devices. Citizens can easily register complaints, track their status, and receive updates in real time, while administrators and departmental authorities access interactive dashboards to manage, assign, and resolve grievances efficiently.

On the backend, Civic-Eye is powered by **Node.js** and **Express.js**, which handle authentication, role-based authorization, API routing, and core business logic. These frameworks provide secure middleware handling and structured RESTful APIs, ensuring that sensitive operations—such as complaint assignment, status updates, and departmental management—are properly controlled and protected based on user roles.

The database layer is implemented using **PostgreSQL**, ensuring robust relational data management and data integrity. Structured models manage entities such as Users, Departments, Complaints, and Status Logs, allowing efficient querying and reliable record maintenance. The integration of PostgreSQL with modern backend architecture ensures consistency, scalability, and secure data handling.

By leveraging these advanced web frameworks and structured development methodologies,

Civic-Eye effectively addresses the inefficiencies of traditional manual grievance systems and delivers a professional and performance-driven digital solution for complaint management.

PROBLEM STATEMENT

4.1 ABSTRACT

Civic Eye is a web-based Grievance Redressal and Complaint Management System designed to digitize and streamline the process of lodging, managing, and resolving public complaints. Traditional grievance systems often rely on manual documentation or fragmented digital tools, leading to delayed responses, lack of transparency, inefficient tracking, and poor accountability across departments.

To overcome these challenges, **Civic Eye** provides a centralized digital platform where citizens can register complaints, track real-time status updates, and receive notifications regarding resolution progress. The system incorporates a structured Role-Based Access Control (RBAC) mechanism, defining three primary user roles: Citizen, Department Officer (sub-Admin), and Super Administrator (super Admin). Each role is granted specific permissions to ensure secure access and operational clarity.

Citizens can log into their personal dashboard to submit grievances, upload supporting documents, and monitor complaint progress. Department Officers are provided with administrative dashboards to review, assign, update, and resolve complaints within their respective departments. The Super Administrator oversees all system activities, manages departments, monitors complaint analytics, and ensures inter-departmental coordination. The system is developed using React.js for building a responsive and interactive frontend interface, Node.js with Express.js for backend API development and business logic implementation, and PostgreSQL for reliable and secure data management. JWT-based authentication ensures secure session handling, while protected routes enforce strict role-based authorization.

By integrating structured workflows, secure authentication, and real-time tracking, **Civic Eye** enhances transparency, improves response efficiency, and promotes accountability within

grievance management systems. The platform bridges the gap between citizens and administrative authorities by delivering a scalable, user-friendly, and technology-driven solution for effective complaint resolution.

4.2 TOOLS AND TECHNOLOGY

The development of **Civic Eye** utilizes a modern and efficient technology stack to ensure scalability, maintainability, and a seamless user experience.

Frontend

- **React.js:** Used to develop a dynamic, component-based user interface. It enables responsive dashboards for citizens and administrators, real-time updates, and modular UI development.
- **Tailwind CSS:** A utility-first CSS framework that ensures responsive design, consistency, and clean user interface styling across devices.

Backend

- **Node.js with Express.js:** Handles server-side logic, API development, routing, middleware management, and authentication. It processes complaint creation, assignment, updates, and user management operations.

Database and Data Access

- **PostgreSQL:** A robust, open-source relational database system used to securely store user credentials, complaint records, departmental data, and status logs. It ensures data consistency and relational integrity.

Development Tools

- **JetBrains WebStorm** – Primary IDE used for frontend and backend development
- **Git & GitHub** Used for version control, collaborative development, and maintaining structured project history.

Hosting & Deployment

- **Neon/ Aiven** – Cloud-hosted PostgreSQL database service ensuring scalability and automated backups.
- **Render / Vercel (for frontend & backend)** – Used for deploying frontend and backend services with continuous integration and deployment support.

Additional Services

- **Node mailer:** Integrated to send automated notifications and complaint status updates to users.
- **JSON Web Tokens (JWT):** Implements secure authentication and session management.
- **Role-Based Middleware:** Custom middleware to restrict API routes based on user roles (Citizen, sub-Admin, super Admin).

4.3 METHODOLOGY

The development of **Civic Eye** follows the Incremental Software Development Model, enabling the system to be built in structured phases with continuous testing and refinement.

Step 1 – Requirement Analysis

- Defined functional requirements such as complaint registration, complaint tracking, department assignment, status updates, and administrative monitoring.
- Established non-functional requirements including security, scalability, transparency, and responsiveness.
- Identified three core user roles: Citizen, sub-Admin (Department Officer), and super Admin

Step 2 – System Design

- Developed UI wireframes for citizen dashboards and administrative control panels.
- Designed a relational database schema including entities such as User, Department, Complaint, Status Log, and Attachment.
- Created ER diagrams and workflow diagrams for complaint submission, assignment, escalation, and resolution processes.

Step 3 – Incremental Development

- **Phase 1:** Implemented user authentication and RBAC system using JWT.
- **Phase 2:** Developed complaint registration and tracking modules.
- **Phase 3:** Built department management and complaint assignment functionalities.
- **Phase 4:** Implemented dashboards for sub-Admin and super Admin users.
- **Phase 5:** Integrated email notification services and complaint analytics.

Step 4 – Testing

- Conducted unit testing for authentication and complaint controllers.
- Performed integration testing between frontend, backend, and PostgreSQL database.
- Executed User Acceptance Testing (UAT) to validate workflows for all user roles.

Step 5 – Deployment

- Deployed database services using cloud PostgreSQL hosting.
- Hosted frontend and backend services using Render/Vercel.
- Configured CI/CD pipelines for smooth updates and maintenance.

Step 6 – Maintenance and Future Scope

- Continuous monitoring of performance and logs.
- Planned enhancements include AI-based complaint categorization, sentiment analysis, and automated complaint prioritization.

By following this methodology, **Civic-Eye** ensures a structured, reliable, and scalable development process that balances usability, automation, and system performance.

4.4 IMPLEMENTATION

The implementation of **Civic Eye** is modular, ensuring scalability and maintainability.

1. Authentication and Role-Based Access Control (RBAC)

- **User & Role Management:** The User model stores credentials and role information. JWT is used for secure login and session validation.
- **Secure Endpoints:** Custom middleware restricts access to routes based on roles:
 - Citizens: Submit and track complaints
 - Sub Admin: Manage complaints within assigned departments
 - Super Admin: Full system access

2. Complaint Management Module

- **Complaint Registration:** Citizens can submit complaints by selecting a department, entering details, and optionally uploading supporting documents.
- **Complaint Tracking:** Each complaint has a unique ID and status stages such as:
 - Pending
 - Assigned
 - In Progress
 - Resolved
 - Rejected
- **Status Logs:** Every status update is recorded in a Status Log table to maintain transparency and audit trails.

3. Department Management Module

- **Department Creation:** Super Admin can create, update, and delete departments.
- **Officer Assignment:** Complaints are assigned to sub-Admins based on department specialization.

4. Administrative Dashboard

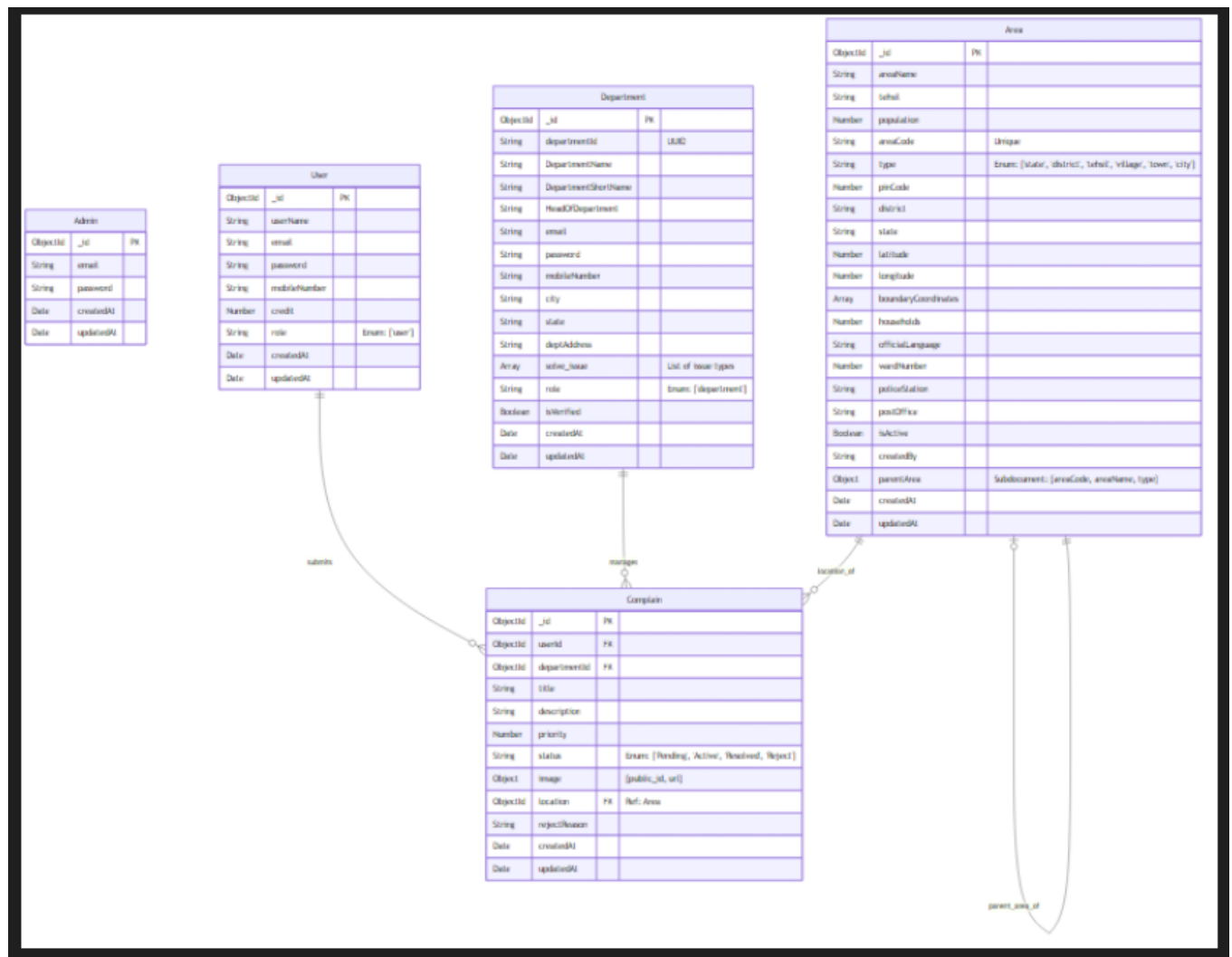
- **Sub Admin Dashboard:** View, assign, update, and resolve complaints within their department.
- **Super Admin Dashboard:** Monitor overall system performance, manage departments, view analytics, and oversee all complaints.

5. System Integration

- **Frontend–Backend Communication:** RESTful APIs built using Express.js handle all data interactions. React.js consumes these APIs to dynamically render dashboards and complaint lists.
- **Database Layer:** PostgreSQL manages relational data including Users, Complaints, Departments, and Status Logs. Foreign key constraints maintain data consistency and integrity.

APPENDICES

1.Database Design:



2.Backend Endpoints:

JS index.js X

Server > JS index.js > ...

```
1  import express from "express";
2  import dotenv from "dotenv";
3  import cors from "cors";
4  import connectDB from "../config/db.js";
5  import bodyParser from "body-parser";
6
7  const app = express();
8  dotenv.config();
9  app.use(cors());
10 app.use(bodyParser.json({ limit: "50mb" }));
11 app.use(bodyParser.urlencoded({ limit: "50mb", extended: true }));
12
13
14
15 import deptRoute from "../Routes/DepartmentRoutes.js";
16 app.use("/department", deptRoute);
17
18 import route from "../Routes/userRoutes.js";
19 app.use("/user", route);
20
21 import complainRoute from "../Routes/complainRoutes.js";
22 app.use("/complain", complainRoute);
23
24 import refreshRoute from "../Routes/refreshRoutes.js";
25 app.use("/refresh", refreshRoute);
26
27 import areaRoute from "../Routes/areaRoutes.js";
28 app.use("/area", areaRoute);
29
30 import adminRoute from "../Routes/adminRoutes.js";
31 app.use("/admin", adminRoute);
32 | Ctrl+I for Command, Ctrl+L for Agent
33 const PORT = process.env.PORT;
34 connectDB().then(() => {
35   app.listen(PORT, () => {
36     console.log(`Server is running on port : ${PORT}`);
37   });
38 });
39
```

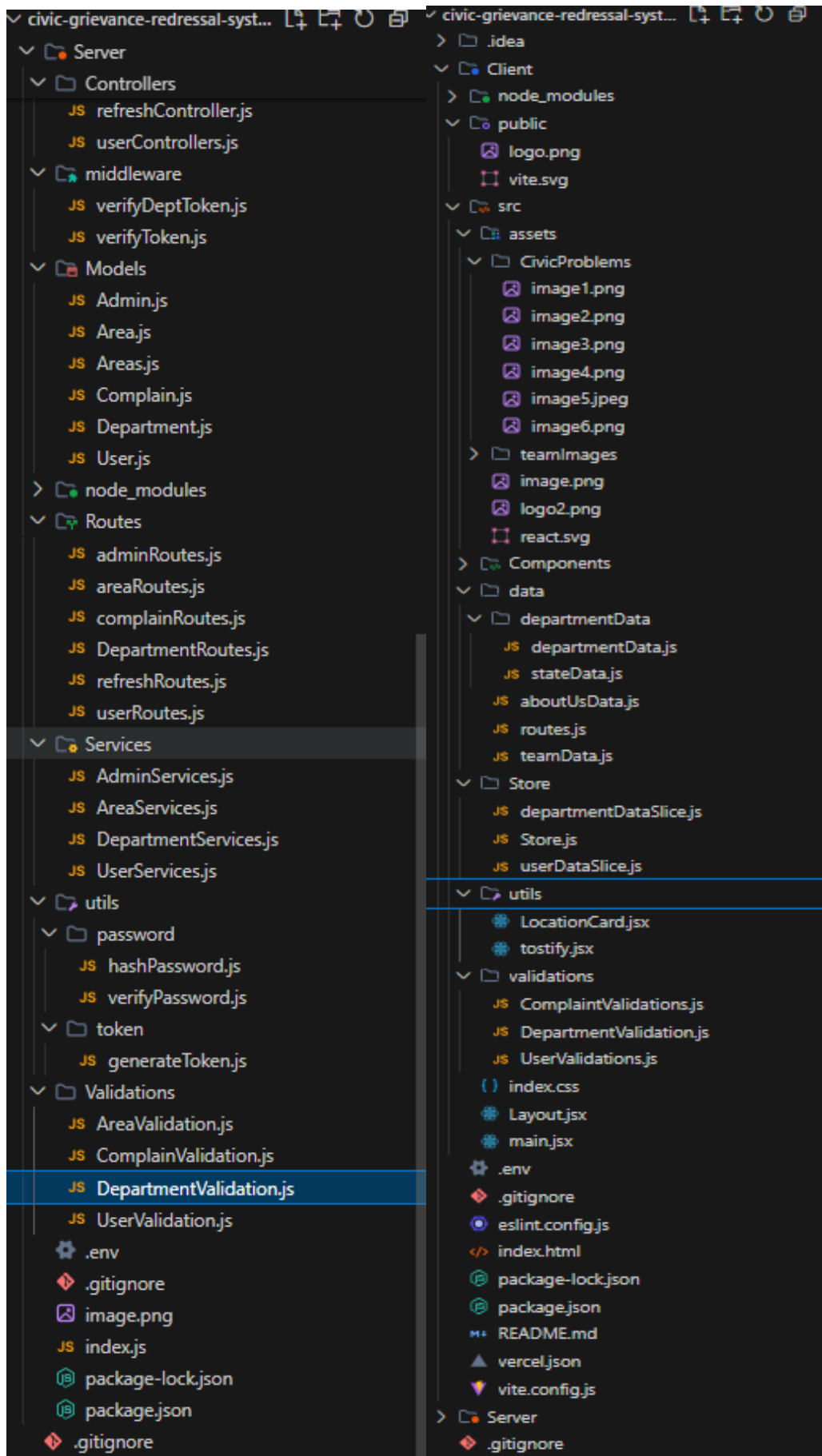
Client > src > main.jsx > ...

```

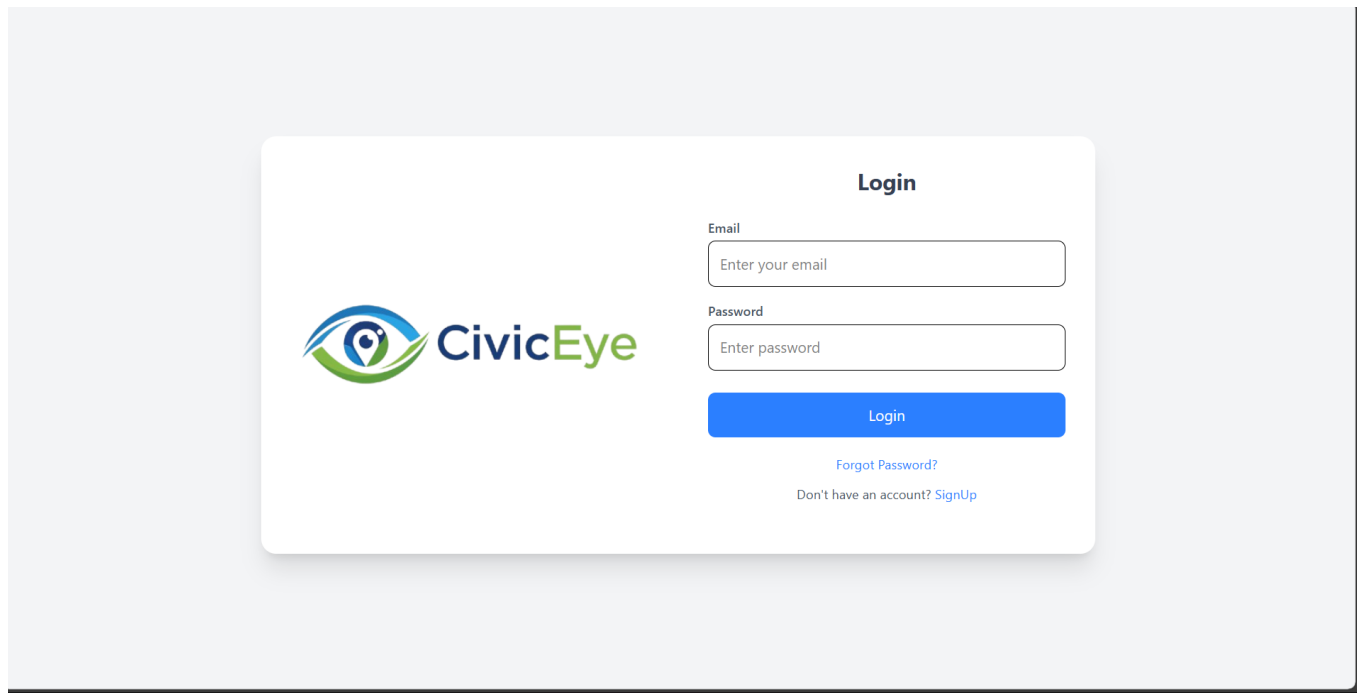
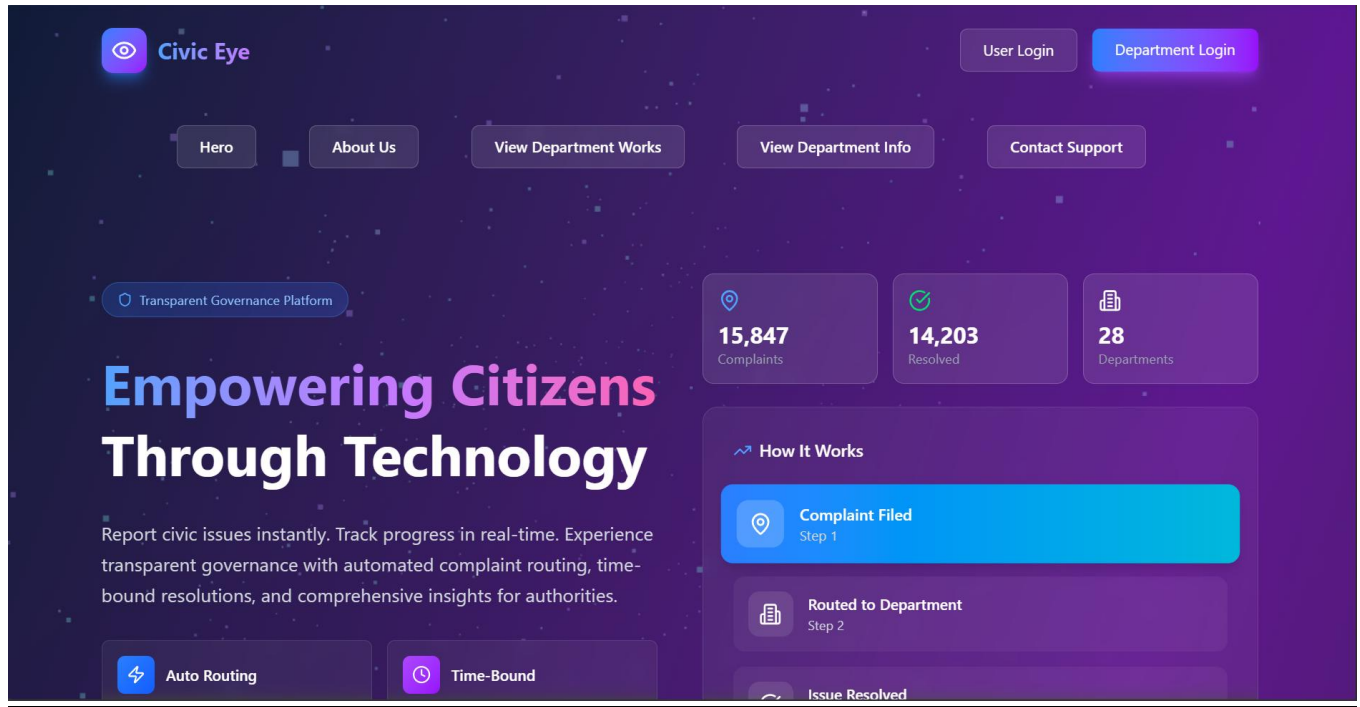
1  import { ToastContainer, toast } from 'react-toastify';
2  import { createRoot } from 'react-dom/client';
3  import { createBrowserRouter, createRoutesFromElements, Route, RouterProvider } from 'react-router-dom'
4  import './index.css'
5  import Layout from './Layout.jsx'
6  import UserSignUpWrapper from './Components/UserSignUp/UserSignUpWrapper.jsx'
7  import { Provider } from 'react-redux'
8  import { Store } from './Store/Store.js'
9  import UserLoginWrapper from './Components/Login/UserLoginWrapper.jsx';
10 import UserDashboard from './Components/Dashboard/UserDashboard.jsx';
11 import { routes } from './data/routes.js';
12 import NotFound from './Components/404/NotFound.jsx';
13 import AboutUs from './Components/screens/AboutUs.jsx';
14 import { Parallax, ParallaxProvider, useParallax } from 'react-scroll-parallax';
15 import Hero from './Components/Hero/Hero.jsx';
16 import OAuth from './Components/auth/OAuth.jsx';
17 import DepartmentSignUpWrapper from './Components/DepartmentSignUp/DepartmentSignUpWrapper.jsx';
18 import DepartmentSignInWrapper from './Components/DepartmentSignIn/DepartmentSignInWrapper.jsx';
19 import DepartmentDashboard from './Components/Dashboard/DepartmentDashboard.jsx';
20 import AllStates from './Components/AllStates/AllStates.jsx';
21 import AllDistricts from './Components/AllDistricts/AllDistricts.jsx';
22 import AllDepartment from './Components/AllDepartment/AllDepartment.jsx';
23 import DepartmentInfo from './Components/DepartmentInfo/DepartmentInfo.jsx';
24 import Header from './Components/Header/Header.jsx';
25 import Footer from './Components/Footer/Footer.jsx';
26 import ComplaintWrapper from './Components/complaint/ComplaintWrapper.jsx'
27 import AdminLogin from './Components/Admin/AdminLogin.jsx';
28 import AdminDashboard from './Components/Admin/AdminDashboard.jsx';
29 import AllComplaints from './Components/AllComplaints/AllComplaints.jsx';
30 import UserComplaint from './Components/userComplaint/UserComplaint.jsx';
31 import DepartmentComplaint from './Components/DepartmentComplaint/DepartmentComplaint.jsx';
32 import ActiveComplaint from './Components/DepartmentComplaint/ActiveComplaint.jsx';
33 import UnverifiedDepartments from './Components/Admin/UnverifiedDepartments.jsx';
34 import UnsolvedComplaints from './Components/Admin/UnsolvedComplaints.jsx';
35 import AdminDashboardHome from './Components/Admin/AdminDashboardHome.jsx';
36 const router = createBrowserRouter(
37   createRoutesFromElements(
38     <>
39       <Route path="/user/*" element={<UserSignUpWrapper />}></Route>
40       <Route path="/user/login" element={<UserLoginWrapper />}></Route>
41
42       <Route path="/department/*" element={<DepartmentSignUpWrapper />}></Route>
43       <Route path="/department/login" element={<DepartmentSignInWrapper />}></Route>
44       <Route path="/department/admin/login" element={<AdminLogin />}></Route>
45
46       <Route path="/admin/*" element={<AdminDashboardHome />}></Route>
47       <Route path="/admin/unverified-departments" element={<UnverifiedDepartments />}></Route>
48       <Route path="/admin/unsolved-complaints" element={<UnsolvedComplaints />}></Route>
49
50       <Route path="/" element={<Hero />}></Route>
51       <Route path="/about-us" element={<Header /><AboutUs /><Footer /></>}></Route>
52       <Route path="/department-info" element={<Header /><AllStates /><Footer /></>}></Route>
53       <Route path="/department-info/:param" element={<Header /><AllDistricts /><Footer /></>}></Route>
54       <Route path="/department-info/:param/:district" element={<Header /><AllDepartment /><Footer /></>}></Route>
55       <Route path="/department-info/:param/:district/:departmentName" element={<Header /><DepartmentInfo /><Footer /></>}></Route>
56
57       <Route path="/auth" element={<OAuth /><Layout /></OAuth>}></Route>
58       <Route path="/user/dashboard" element={<OAuth><UserDashboard /></OAuth>}></Route>
59       <Route path="/department/dashboard" element={<OAuth><DepartmentDashboard /></OAuth>}></Route>
60       <Route path="/department/raise-complaint" element={<OAuth><ComplaintWrapper /></OAuth>}></Route>
61       <Route path="/department/see-complaints" element={<OAuth><AllComplaints /></OAuth>}></Route>
62       <Route path="/user/complaint" element={<OAuth><UserComplaint /></OAuth>}></Route>
63       <Route path="/department/complaint" element={<OAuth><DepartmentComplaint /></OAuth>}></Route>
64       <Route path="/department/active-complaint" element={<OAuth><ActiveComplaint /></OAuth>}></Route>
65
66       <Route path="*" element={<NotFound />}></Route>
67     </>
68   )
69 )
70
71 createRoot(document.getElementById('root')).render(
72   <Provider store={Store}>
73     <ToastContainer />
74     <RouterProvider router={router} />
75   </Provider>
76 )

```

```
JS index.js  Layout.jsx X
Client > src > Layout.jsx > ...
1  import { Outlet } from 'react-router-dom'
2  import Footer from './Components/Footer/Footer'
3  import Header from './Components/Header/Header'
4  import OAuth from './Components/auth/OAuth'
5  import { useSelector, useDispatch } from "react-redux";
6  import UserSidebarNavigation from './Components/SidebarNavigation/UserSidebarNavigation'
7  import DeptSidebarNavigation from './Components/SidebarNavigation/DeptSidebarNavigation'
8  function Layout() {
9
10     const userData = useSelector((store) => store.userData);
11     const deptData = useSelector((store) => store.departmentData);
12     const userRole = userData?.role;
13     const deptRole = deptData?.role;
14     return (
15     <>
16         <Header />
17         {userRole !== "" || deptRole !== "" ?
18             userRole !== "" ?
19             <div className={'flex gap-7'}>
20                 <OAuth> <UserSidebarNavigation /></OAuth>
21                 <Outlet />
22             </div>
23             :
24             <div className={'flex gap-7'}>
25                 <OAuth> <DeptSidebarNavigation /></OAuth>
26                 <Outlet />
27             </div>
28             : ''
29         }
30         { /* <Outlet/> */ }
31         <Footer />
32     </>
33     )
34 }
35
36 export default Layout
```



3. Frontend Full-UI:



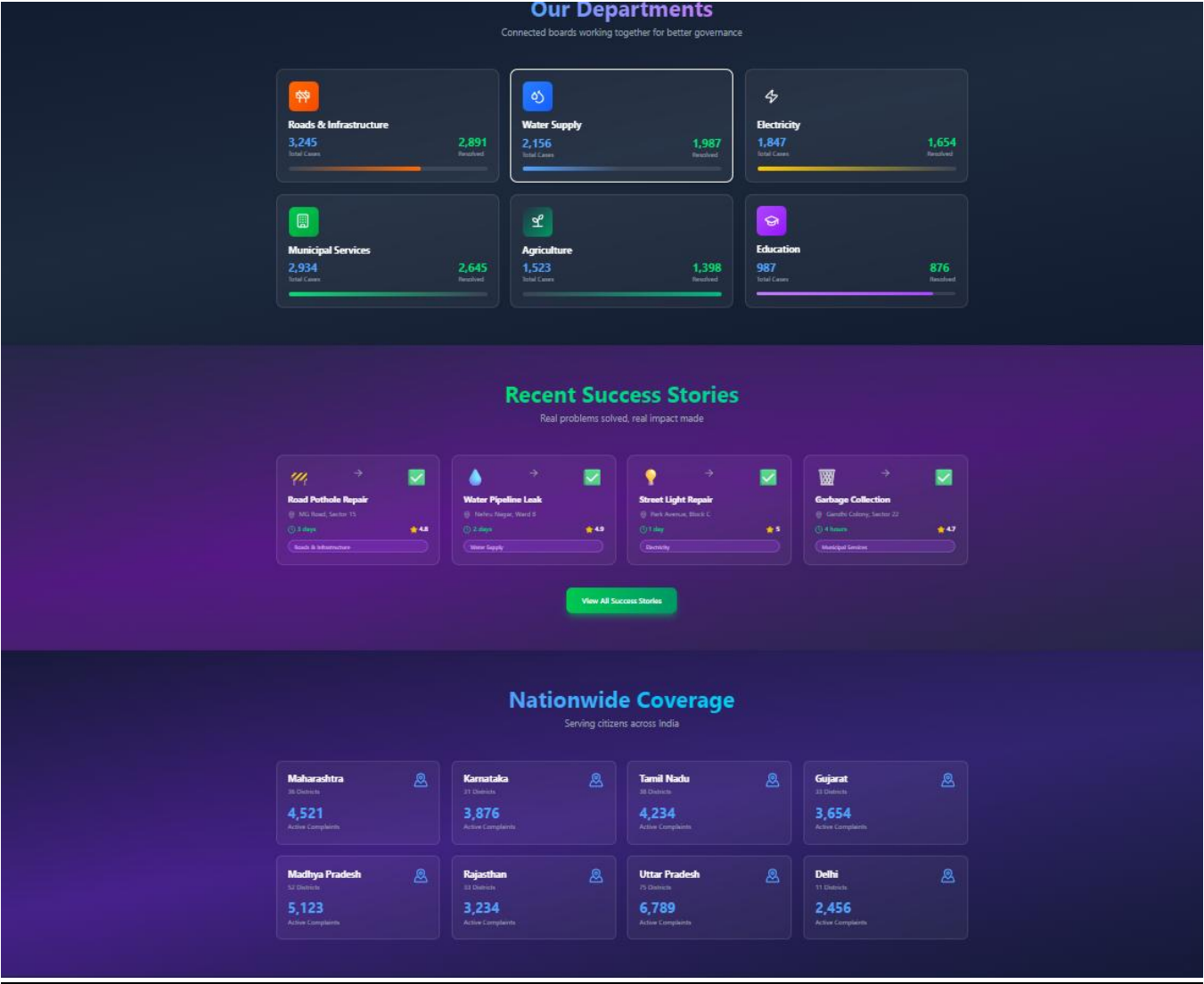
Me



**Have A Problem To Report?
Your Voice Matters For
Change.**

Help us build a better community by reporting issues





 **Department Sign Up**

1

2

3

Step 1 of 3: Department Information

Department Name

Enter department name

Head of Department

Enter HOD name

Department Short Name

e.g., PWD, EDU

Previous

Next

Already have an account? [Sign In](#)

 **Department Sign Up**

1

2

3

Step 3 of 3: Address & Security

State

Goa

State name is required

District

Select District

District name is required

Department Address

Office address

Department Address name is required

Password

Create password

Password is required

Previous

Submit

Already have an account? [Sign In](#)



Create Account

UserName

Email

Send
OTP

OTP

Mobile Number

Password

Confirm Password

Submit

Already have an account? [Sign In](#)